



5cpu-scheduling

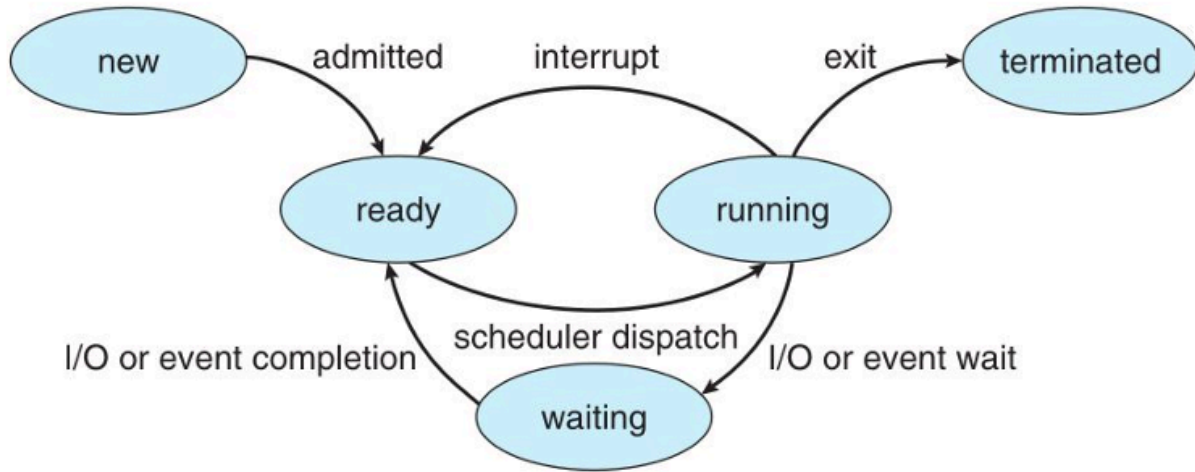
CPU Yönetiminin Temelleri

CPU Zaman Yönetiminin Hedefleri

- CPU'yu her zaman meşgul tutmak.
- Her process'e adil davranmak.
- Kullanıcıya hızlı yanıt vermek.

Scheduling Triggers

- **Çalışan (Running) → Bekleyen (Waiting):** Örneğin, çalışan program klavyeden bir girdi bekliyorsa veya bir dosya okuyorsa boşuna işlemciyi meşgul etmez, beklemeye geçer.
- **Çalışan (Running) → Hazır (Ready):** Programın çalışma süresi dolmuştur, işletim sistemi onu durdurup sıraya geri koyar.
- **Bekleyen (Waiting) → Hazır (Ready):** Programın beklediği dosya inmiştir veya kullanıcı klavyeye basmıştır, program tekrar çalışmaya hazır hale gelir.
- **Sonlanma (Terminates):** Program işini tamamen bitirip kapanmıştır.



Preemptive & Non-Preemptive

- **Non-Preemptive (Kesintisiz):** Bir process cpu'yu aldıysa bitene kadar (ya da waiting olana kadar) geri vermez.
- **Preemptive (Kesintili):** İşletim sistemi process'i zorla durdurup cpu'yu başka bir process'e verebilir.

CPU-I/O Burst Döngüsü

Process'ler CPU hesaplamaları ve I/O işlemleri arasında gidip gelerek çalışırlar. CPU'da yaptıkları hesaplamalara **CPU burst**; internetten bir şey indirmek, dikten bir şey okumak gibi işlemlere de **I/O burst** denir.

CPU burst bu döngünün çok küçük bir parçasıdır, bu yüzden CPU bu döngüyü takip ederek scheduling yapar.

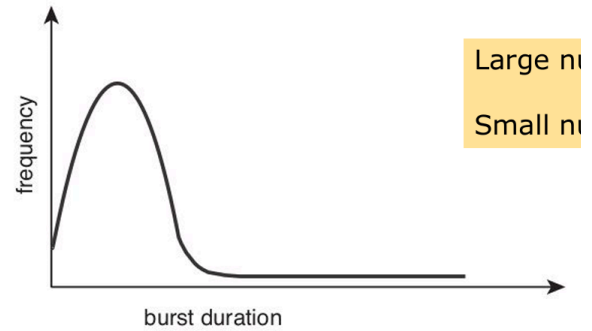
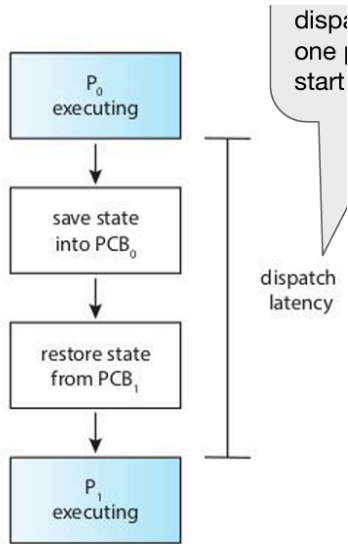


Figure 5.2 Histogram of CPU-burst durations.

Scheduler & Dispatcher

- **CPU Scheduler (Çizelgeleyici):** Hazırda bekleyen programlar arasından "Sıradaki bu olsun" diye sadece **seçim yapar**.



- **Dispatcher (Dağıtıcı):** Seçilen programı **gerçekten alıp işlemciye oturtan** ve çalışmasını başlatan modüldür.
 - Dağıtıcı bu değişimi yaparken eskisinin nerede kaldığını kaydeder, yenisinin bilgilerini yükler.
 - Buna "Context Switch" (Bağlam Değişimi) denir. Bu değişim sırasında geçen süreye ise **Dispatch Latency (Dağıtım Gecikmesi)** denir.
 - İşletim sistemi bu gecikmenin çok ama çok kısa olmasını ister, çünkü bu sürede işlemci hiçbir işe yaramaz, sadece geçiş işlemleriyle uğraşır.

Scheduling Algorithms

CPU Scheduling Algoritmalarının Kriterleri

Algoritmaları karşılaştırırken 5 temel kriterden yararlanılır.

- **CPU Kullanımı (Utilization):** CPU'yu olabildiğince meşgul tutmak. (Bunu her zaman maksimize etmeye çalışırız).
- **Verimlilik:** Birim zamanda mümkün olduğunca çok iş bitirmek. (Maksimize etmeye çalışırız).
- **Geri Dönüş Süresi:** Süreçlerin baştan sona bitme süresini en aza indirmek. (Minimize etmeye çalışırız).
- **Bekleme Süresi:** Programların sırada beklediği süreyi en aza indirmek. (Minimize etmeye çalışırız).
- **Yanıt Süresi:** Sistemin kullanıcıya yanıt verme süresi en aza indirmek. (Minimize etmeye çalışıyoruz).

Scheduling Algorithms

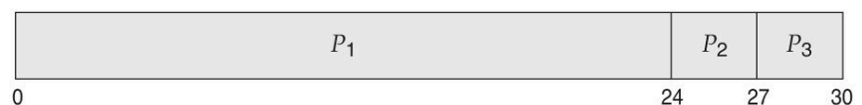
First- Come, First-Served (FCFS) Scheduling

- İlk giren ilk çıkar ve işi bitmeden diğerine geçilmez.
- I/O Wait olduğunda işlemciyi bırakır.
- Non-Preemptive bir algoritmadır.

Process	Burst Time
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1, P_2, P_3

- The **Gantt Chart** for the schedule is



- Average waiting time hesabı aşağıdaki gibi yapılır:
 - Waiting times
 - $WTP_1 = 0$
 - $WTP_2 = 24$
 - $WTP_3 = 27$
 - Average waiting time: $AWT = (0 + 24 + 27)/3 = 17$

Uzun process'ler için iyidir ancak kısa process'ler için kötüdür, en iyi performansı kısıdan uzuna sıralandıklarında verirler.

Shortest-Job-First (SJF) Scheduling

- Hazırda bekleyen programlar arasından, işlemcideki (CPU) sıradaki işi **en kısa sürecek olanı** seç ve önce onu çalıştır.
- Teorik olarak, belirli bir süreç grubu için ortalama bekleme süresini **minimuma indiren (optimal) algoritma SJF'dir**.

- SJF algoritması sonsüz döngüleri yönetemez.

Process	Arrival Time	Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

P_1	P_2	P_4	P_1	P_3
0	1	5	10	17
26				

- Average waiting time = 6.5
 - non-preemptive SJF awt = 7.75
- Bir sürecin sonsuza kadar ertelenip işlemciye hiç çıkamaması durumuna işletim sistemlerinde **Starvation (Açlık)** denir. SJF, uzun işlemlerin açlıktan ölmesine sebep olabilir. Eğer sisteme sürekli olarak 1-2 milisaniye sürecek kısacık görevler gelmeye devam ederse, 30 milisaniye sürecek olan o "uzun" program kuyruğun en arkasında kalır ve **belki de işlemciyi hiçbir zaman kullanamaz**.

CPU Burst Tahmin Etme

SJF kağıt üzerinde mükemmeldir ama gerçek hayatta devasa bir zorluğu vardır: İşletim sistemi, sıradaki programın işlemcide **ne kadar süre kalacağını (CPU Burst) önceden kesin olarak bilemez**. Bunu çözmek için işletim sistemi "Tahmin Yürütme" (Exponential Averaging) yapar. Programın geçmişteki çalışma sürelerine bakarak, bir sonraki gelişinde ne kadar süreceğini tahmin etmeye çalışır ve bu tahminlere göre sıraya koyar.

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$$

- τ_{n+1} (Yarınki Tahminimiz): Bir sonraki sefer bu program işlemciye geldiğinde işinin ne kadar süreceğine dair yapacağımız yeni tahmin. PDF + 1
- t_n (Bugünkü Gerçek Durum): Programın işlemcide *gerçekte* ne kadar süre harcadığı (En son CPU patlaması). PDF + 2
- τ_n (Dünkü Tahminimiz): Geçmişteki tahminimiz. PDF + 1
- α (Güven Katsayısı): 0 ile 1 arasında bir sayıdır. "En son olan olaya (gerçeğe) ne kadar güvenmeliyim?" sorusunun cevabıdır. PDF

Genelde α değeri 0.5 olarak alınır (yarı yarıya). Bu şu demektir: "Yeni tahmini yaparken, az önce gerçekten ne olduğuna %50 ağırlık ver, geçmişten gelen eski tahminlerime de %50 ağırlık ver." Böylece program aniden çok uzun süren bir işlem yaparsa işletim sistemi hemen panikleyip tahminini tamamen değiştirmez, yavaş yavaş yeni duruma adapte olur.

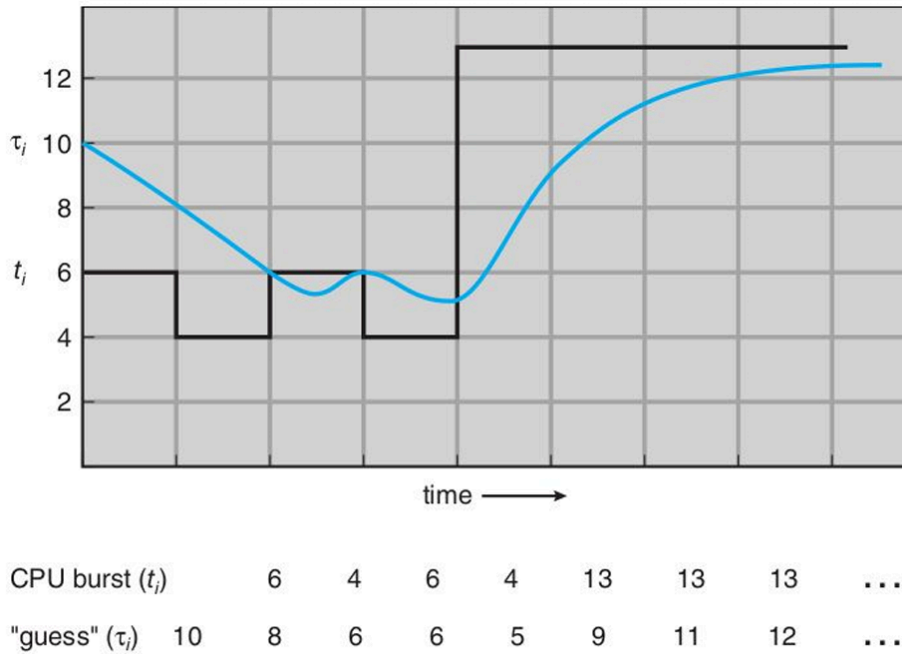
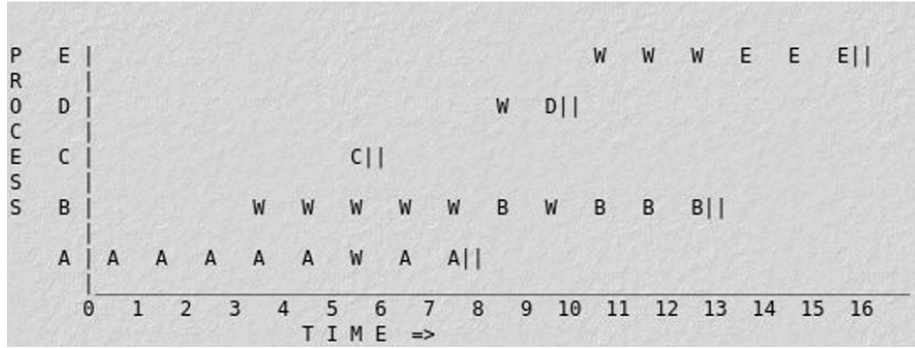


Figure 5.4 Prediction of the length of the next CPU burst.

Shortest-Remaining-Time (SRT) Algorithm

- SRT, SJF'nin **Preemptive (Kesintili/Zorla alınabilen)** versiyonudur.

- SJF algoritmasından farkı, mevcutta çalışan process'in kalan süresi eğer yeni gelenden fazlaysa mevcut process'i yarıda keser ve daha kısa olanı alır.



- **Çok Kısa İşler İçin Harikadır:** Kısa işlemler anında sisteme girip çıktığı için çok iyi hizmet alırlar.
- **Verimlilik Kralı:** Tahminler tam olarak doğruysa, tüm algoritmalar arasında verimliliği (biten süreç sayısını) en üst düzeye çıkaran (optimal) algoritmadır.
- **Açlık (Starvation) Tehlikesi:** Tıpkı SJF gibi, eğer sisteme sürekli kısıp işler gelmeye devam ederse, o 500 sayfalık kalın kitabın (uzun işlerin) kapağını bile açamayabilirsin. Uzun süreçler açlıktan ölebilir.

Round Robin (RR) Algorithm

- Her process'i belirli bir süre çalıştırıp diğerine geçer.
- Baştan sona bitirme süresi (Turnaround time) SJF algoritmasına göre genellikle daha kötüdür.
- RR'nin tek seferde process çalıştırma süresi (quantum) belirlenirken genelde işlemci görevlerinin (CPU Burst) %80'inden daha uzun olmalıdır.
- Quantum genelde 10 milisaniyeyle 100 milisaniye arasında seçilir, context switch 10 milisaniyeden daha kısadır.

Priority Scheduling

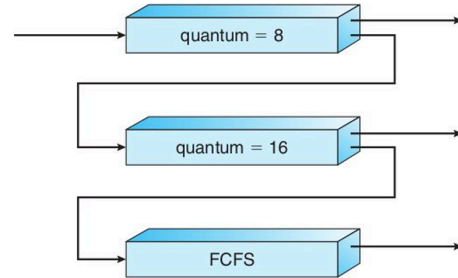
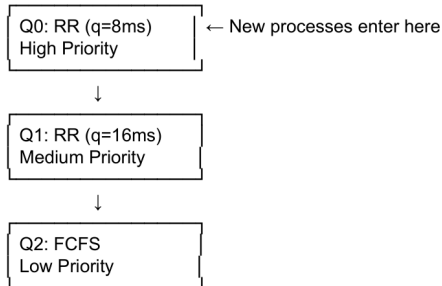
OS her process'e bir öncelik numarası atar. Genelde sayı küçüldükçe öncelik artar.

- Burada da diğerlerinde olduğu gibi starvation yaşanabilir ve alt queue'lara asla sıra gelmez.

Çözüm: Çok Seviyeli Geri Beslemeli Kuyruk (MLFQ)

- Bu durumda demotion metodu uygulanır.
 - Bu metoda göre her process önce en yüksek öncelikli queue'ya girer.
 - Kısa ve anlık işlemler $q = 8ms$ olan bu queue'da işini tek seferde halledip gider.
 - Uzun işlemlerse ilk seferde işlemini tamamlayamadığı için $q = 16ms$ olan 2. queue'ya atılır. Eğer orada da işlemini tamamlayamazsa bir alt queue'ya gönderilir.
 - Eğer ağır işlemler en alt kuyrukta (Q2) çok uzun süre beklerse, sistem onlara acır ve **Yaşlandırma (Aging)** uygulayarak onları tekrar en üst kuyruklara (Q0 veya Q1) terfi ettirir.

The Best of All Worlds: Separate queues for each priority



Thread Scheduling

Thread'ler için 2 farklı rekabet alanı (scope) vardır:

1. Process İçi Rekabet (PCS)

- Burada rekabet sadece **aynı programın içindeki thread'ler arasındadır**.
- Bir process işlemciyi aldığı anda hangi process içi thread'in işlemciyi kullanacağına OS değil process karar verir.

2. Sistem Geneli Rekabet (SCS)

- Rekabet, **bilgisayardaki tüm thread'ler** arasındadır.
- İşletim sistemi çekirdeği (Kernel) olaya doğrudan el atar.
- Her process'in thread'i arenada kozlarını paylaşırlar.
- İşletim sistemi hepsini tek bir havuzda görür ve önceliklerine göre işlemciye atar.

Multiple-Processor Scheduling

1. Kuyruk Mimarisi

Birden fazla işlemcimiz varsa, çalışmayı bekleyen süreçleri nerede tutacağız? Simetrik Çoklu İşleme (SMP) sistemlerinde iki temel yaklaşım vardır:

1. **Ortak Kuyruk:** Tıpkı bankadaki tek bir numaratör kuyruğu gibidir. Tüm işlemciler boşa çıktıkça gelip bu ortak havuzdan iş çekerler. Mantıklı görünse de, birden fazla işlemci aynı anda aynı sıradan iş çekmeye kalkarsa "kilitlenme" (lock) sorunları yaşanır ve performans düşer.
2. **Çekirdek Başına Kuyruk:** Süpermarketlerdeki her kasanın kendi kuyruğu olması gibidir. Her işlemcinin kendine ait, sadece onun ilgilendiği bir kuyruğu vardır. Modern sistemlerde en çok kullanılan yöntem budur çünkü kilitlenme sorunu yaratmaz.

2. Yük Dengeleme

Eğer her çekirdeğin kendi kuyruğu varsa, ortaya yeni bir sorun çıkar: Ya 1. çekirdeğin kuyruğunda çok ağır 10 tane oyun süreci birikirse ve 2. çekirdeğin kuyruğu bomboş kalırsa? İşletim sistemi adaleti sağlamak için yük dengelemesi yapar. İki taktiği vardır:

1. **İtme (Push Migration):** Sistem düzenli olarak çekirdekleri kontrol eder. Eğer bir çekirdek aşırı yüklüyse, onun kuyruğundaki işleri zorla alır ve boş yatan çekirdeklerin kuyruğuna iter.

2. **Çekme (Pull Migration):** Boşta kalan, yapacak işi olmayan bir çekirdek, "Benim canım sıkıldı" diyerek gider ve aşırı yoğun olan çekirdeğin kuyruğundan kendine iş "çeker".

3. İşlemci Yakınlığı

İşletim sistemi sırf yükü dengelemek için programları sürekli bir çekirdekten alıp diğerine atmayı **hiç sevmez**. Çünkü bir program bir çekirdekte bir süre çalıştığında, o çekirdeğin **Önbelleği (Cache)** o programın verileriyle dolar. Sen bu programı alıp başka bir çekirdeğe atarsan, verilerin RAM'den baştan yüklenmesi gerekir ve bu büyük bir zaman kaybıdır. Buna **İşlemci Yakınlığı / Bağımlılığı (Processor Affinity)** denir.

- **Soft Affinity (Yumuşak Yakınlık):** İşletim sistemi "Seni elinden geldiğince aynı çekirdekte tutmaya çalışacağım ama gerekirse başka yere de taşıyabilirim, söz vermiyorum" der.
- **Hard Affinity (Sert Yakınlık):** Programın kodunda kesin bir kural vardır. "Ben sadece 1. ve 2. çekirdeklerde çalışırım, beni başka yere taşıyamazsın" diye sisteme emir verir.

4. Bellek Gecikmesi ve Hyperthreading (Memory Stall)

İşlemci inanılmaz hızlıdır ancak RAM (Bellek) ona göre çok yavaştır. İşlemci RAM'den bir veri istediğinde, o veri gelene kadar boş boş beklemek zorunda kalır. Buna **Memory Stall (Bellek Duraksaması)** denir.

İşlemcilerin boş yere yatmasını engellemek için donanım mühendisleri bir fiziksel çekirdeğin içine birden fazla "Donanımsal İş Parçacığı" (Hardware Thread) koymuşlardır (Intel buna Hyperthreading der). Böylece Thread 1 bellekten veri beklerken (Memory Stall yaşarken), çekirdek saniyesinde Thread 2'ye geçer ve hesaplama yapmaya devam eder; donanım asla boş durmaz.

Real-Time Scheduling

Zamanın Kritik Olduğu Sistemler

Frene bastığın an (olay) ile bilgisayarın bunu işleme alması arasında geçen süreye **Olay Gecikmesi (Event Latency)** denir. İşletim sisteminin eski işi yaka paça kenara atıp (Dispatch Latency), frene basma kodunu işlemciye alması çok ama çok hızlı olmalıdır.

1. **Yumuşak Gerçek Zamanlı Sistemler:** En yüksek önceliğe sahiptirler ama tam zamanında bitme gibi kesin bir garantileri yoktur.
Örneğin, canlı video yayını. Video 1 saniye takılırsa kalite düşer ama sistem tamamen çökmez.
2. **Sert Gerçek Zamanlı Sistemler:** Süreç, verilen süre (deadline) içinde **kesinlikle** bitirilmek zorundadır. Uçak otopilotu veya araba freni gibi.

Planlama

Gerçek zamanlı sistemlerde görevler genellikle **periyodiktir (düzenli tekrar eder)**. Her görevin ne kadar süreceği (t), hangi süreye kadar bitmesi gerektiği (d - deadline) ve ne sıklıkla tekrarlandığı (p - period) bellidir. Bu tarz hayati sistemler için iki ana algoritma kullanılır:

1. **Rate Monotonic (RM) Algoritması:**

Bu algoritma öncelikleri tamamen sabittir, baştan belirlenir ve asla değişmez.

Kuralı çok basittir: **Bir iş ne kadar sık tekrarlanıyorsa (periyodu ne kadar kısaysa), önceliği o kadar yüksektir.**

Mantığı: Saniyede 100 kere kontrol edilmesi gereken bir sensörün, dakikada 1 kere kontrol edilecek bir ısı ölçere göre çok daha acil işlemciye ihtiyacı vardır.

2. **Earliest Deadline First Algoritması:**

Bu algoritma RM'nin aksine dinamiktir, öncelikleri anlık olarak değiştirir.

Kuralı şudur: **O an teslim tarihine (deadline) en yakın olan iş kiminse, öncelik ondadır.**

Mantığı: Üniversitedeki ödevler gibi düşün. Hangi dersin teslimi yarınsa bütün odağını (işlemciyi) ona verirsin. Teslimi haftaya olan ödevi bekletirsin. İşlemci gücünü %100'e kadar mükemmel kullanan en iyi (optimal) yöntemdir.

Teslim Tarihi Aşılırsa Ne Olur?

- **Soft Sistemlerde:** İşlem gecikmeli de olsa devam eder (çünkü geç gelen video karesi, hiç gelmemesinden iyidir).
- **Hard Sistemlerde:** Eğer frene basmak için ayrılan süre geçildiyse, artık o işlemi devam ettirmenin bir mantığı veya güvenliği kalmamıştır. İşlem anında **iptal edilir (abort)** ve hata/kurtarma mekanizması devreye girer.

Gerçek Dünya Örnekleri

1. Linux

Linux, eski sürümlerinde farklı algoritmalar denedikten sonra çok zekice bir sisteme geçiş yaptı: **Tamamen Adil Çizelgeleyici (CFS)**.

CFS'nin mantığı şudur: İdeal bir dünyada işlemcinin gücü, sırada bekleyen herkese eşit ve anında bölünmelidir. Bunu yapabilmek için klasik "Zaman Dilimi (Quantum)" mantığını bir kenara bırakır ve **vruntime (Sanal Çalışma Süresi)** adında bir değişken kullanır.

- **Kural çok basittir:** O an sistemde **vruntime** değeri en küçük olan (yani o ana kadar işlemciden en az faydalanmış olan, en çok hakkı yenen) program hangisiyse, işlemciye o geçer.
- İşletim sisteminin bu "en az çalışanı" binlerce program arasından anında bulabilmesi için veriler **Red-Black Tree (Kırmızı-Siyah Ağaç)** adı verilen özel bir ağaç yapısında tutulur. Bu ağacın en solundaki yaprak, her zaman işlemciyi en çok hak eden programdır ve çekip alınması çok hızlıdır.

2. Windows

Windows, 32 seviyeli (0-31 arası) ve her an araya girilebilen (preemptive) bir öncelik sistemi kullanır.

- **16 ile 31 arası:** Uçak simülatörleri veya ses işleme gibi "Gerçek Zamanlı" (Real-time) işlere ayrılmıştır.
- **1 ile 15 arası:** Oyunlarımız, tarayıcılarımız ve günlük uygulamalarımız içindir (Variable class).

- **Kritik Hile:** Windows'un bilgisayarı bize "akıcı" hissettirmesinin çok güzel bir sırrı vardır. Ekranda o an aktif olarak kullandığın, üzerine tıkladığın pencereye (Ön Plan / Foreground) normalin **3 katı öncelik takviyesi (boost)** yapar. Yani arka planda inen oyun umursanmazken, fareyle tıkladığın Chrome sekmesi anında işlemciyi kapar.

3. Solaris (Oracle'ın İşletim Sistemi)

